Despliegue de app en infraestructura

Descripción del proyecto:

Consiste en una aplicación de reseñas de películas desarrollada con Node.js, compuesta por un frontend que opera en el puerto 3030 y un backend en el puerto 3000. El backend se encarga de la lógica de negocio y de la comunicación con una base de datos MySQL, donde se almacenan las reseñas.
La inicialización de datos se realiza mediante un script de seed (seed.js), que permite insertar los datos correspondientes en la base de datos de forma automatizada.

Desde una perspectiva DevOps, la aplicación ha sido diseñada para facilitar su despliegue, escalabilidad y mantenimiento. Los distintos componentes del sistema son contenerizados mediante Docker, (frontend y backend en Docker en EC2), lo que garantiza entornos consistentes y reproducibles en todas las etapas del ciclo de desarrollo.

Por otra parte, toda la infraestructura necesaria (servidores, elementos de red como: VPC, rutas, security groups) y base de datos) son gestionados mediante herramientas de Infraestructura como Código como Terraform, permitiendo definir y versionar los recursos en la nube de manera automatizada. Toda la parte de la instalación de paquetes y servicios se realiza con la herramienta Ansible que permite la automatización de las configuraciones y con un inventario dinámico podemos trabajar con diferentes EC2 en un ambiente dinámico como AWS cloud.

Adicionalmente, la aplicación se integra en un flujo de integración y entrega continua (CI/CD), automatizando procesos como la construcción, pruebas y despliegue de la app. Esto permite reducir errores manuales y acelerar la entrega de nuevas funcionalidades.

También se pueden incorporar herramientas de monitoreo y logging para supervisar el rendimiento del sistema y detectar posibles fallos en tiempo real. Todo esto se realiza con el servicio de AWS CloudWatch para servicios de AWS.

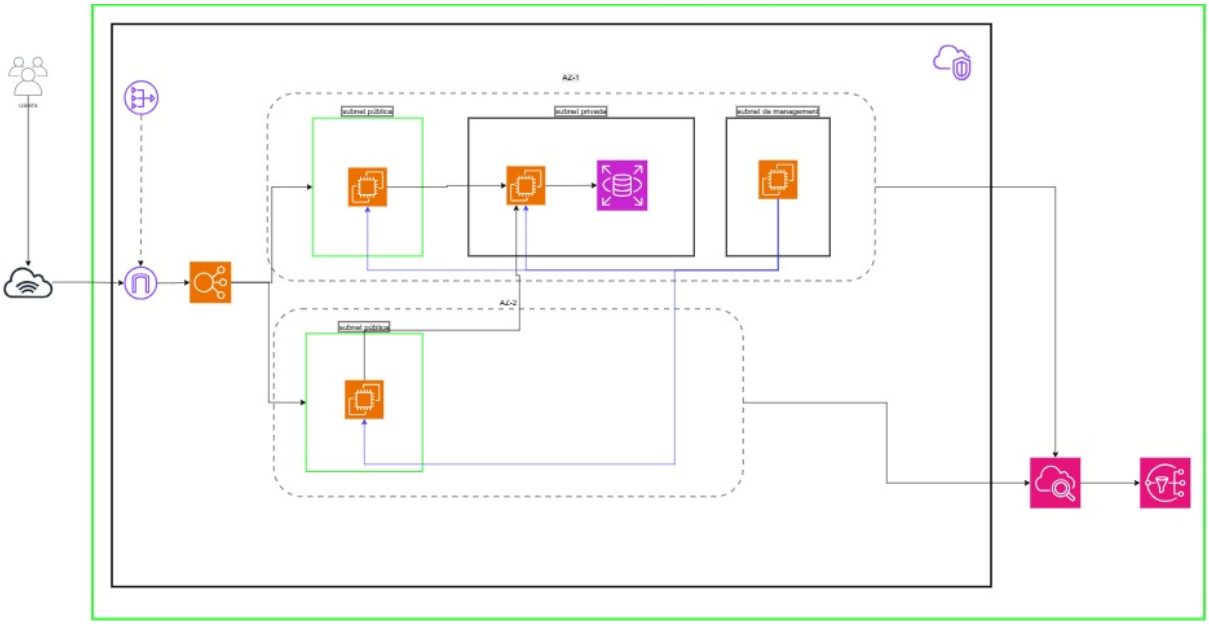
En conjunto, estas prácticas permiten que la aplicación no solo sea funcional, sino también escalable, mantenible y alineada con principios modernos de DevOps.

Paso 1: Crear toda la infraestructura con Terraform

Necesario:
1- Tener las credenciales de AWS en nuestra laptop (aws configure) con un IAM user de administrador.
2- Crear previamente el bucket de terraform donde se guardarán los terraform.tfstate de cada environment. (Seguridad buenas prácticas)
3- Crear los environment (dev, qa, prod)
terraform workspace new dev

```
camila@camila-IdeaPad-3-15ITL6:/media/camila/DATA/PROYECTO OMNICLOUD/terraform$ terraform workspace new dev
You're now on a new, empty workspace. Workspaces isolate their state,
so if you run "terraform plan" Terraform will not see any existing state
for this configuration.
camila@camila-IdeaPad-3-15ITL6:/media/camila/DATA/PROYECTO OMNICLOUD/terraform$ terraform workspace new qa
Created and switched to workspace "qa"!

You're now on a new, empty workspace. Workspaces isolate their state,
so if you run "terraform plan" Terraform will not see any existing state
for this configuration.
camila@camila-IdeaPad-3-15ITL6:/media/camila/DATA/PROYECTO OMNICLOUD/terraform$ terraform workspace new prod
Created and switched to workspace "prod"!

You're now on a new, empty workspace. Workspaces isolate their state,
so if you run "terraform plan" Terraform will not see any existing state
for this configuration.
camila@camila-IdeaPad-3-15ITL6:/media/camila/DATA/PROYECTO OMNICLOUD/terraform$ terraform workspace list
  default
  dev
* prod
  qa

camila@camila-IdeaPad-3-15ITL6:/media/camila/DATA/PROYECTO OMNICLOUD/terraform$ terraform workspace select dev
Switched to workspace "dev".
camila@camila-IdeaPad-3-15ITL6:/media/camila/DATA/PROYECTO OMNICLOUD/terraform$
```

Diagrama de red



Elementos del main.tf:

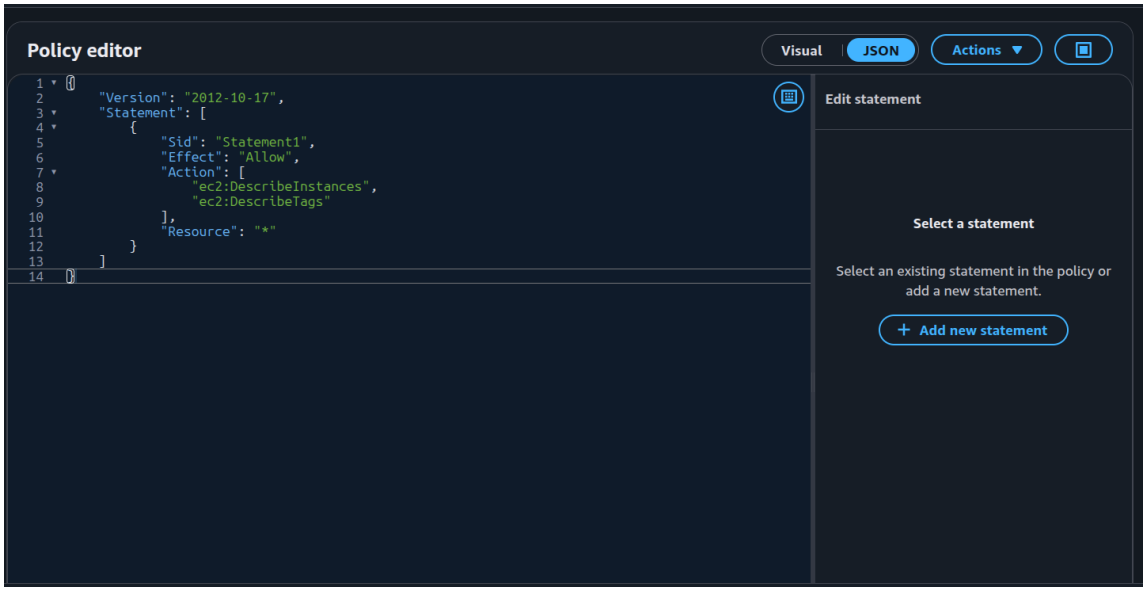
1- Se crean instancias para backend y management. (en la Ec2 de management estará el servidor ansible y Jenkins CI/CD)
2- VPC para la infra con:
SG de frontend, backend, management.
2 subnet publicas donde se encuentran las EC2 de frontend del AutoScaling group
1 subnet privada donde se encuentra la EC2 de backend
1 subnet de administración donde se encuentra el hostjump bastion
Internet gateway para las Ec2 públicas
Nat gateway para las Ec2 privada
Security Groups para cada Ec2 de frontend, backend y management, database y ALB
3- AutoScaling Group para las instancias Ec2 con el template de la ami ami-0c1fe732b5494dc14 que es Amazon Linux 2023
4- Application Load Balancer que redirige el tráfico HTTP del port 80 al ASG
5- Amazon RDS con engine MySQL para las conexiones de base de datos
6- Metricas y alertas con el recurso CloudWatch integradas con el servicio Amazon SNS

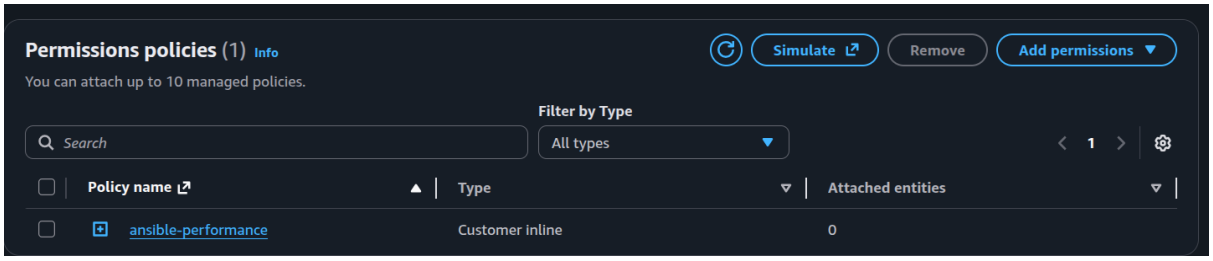
Uso de CPU: >80%
Uso de memoria RAM: >80%
Uso de disco principal: >85%

Métricas cada 5min y con alertas enviadas por el servicio Amazon SNS a la cuenta de email kmilavaut@gmail.com. Para recibir estas alertas se debe confirmar la suscripción en el email. te mandan un correo de validación.

Las métricas de AWS CloudWatch solo monitorean a las instancias de ASG al ser dinámicas, para el caso del backend y otras EC2 se monitorean por otra herramienta (Prometheus).

A las instancias de EC2 se les asigna un rol previamente creado que tiene las políticas de: DescribeInstance y DescribeTags para el funcionamiento del inventario dinámico de ansible



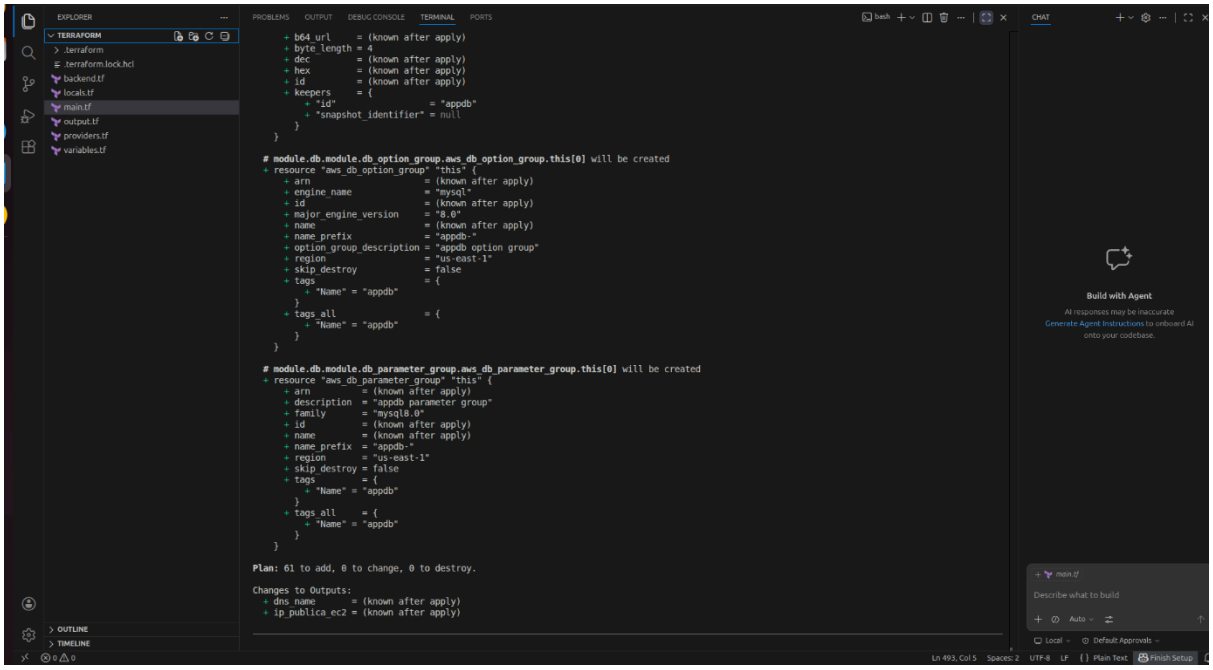


Comandos:

terraform init
terraform workspace show

export TF_VAR_dbusername=applicationuser
export TF_VAR_dbpassword=applicationuser

terraform plan
terraform apply –auto-approve



Paso 2: Configuración de los paquetes y servicios de los servidores. Utilizar Ansible

Necesario:

1- El server hostjump será nuestro bastión que permitirá la administración de configuración del resto de los servers, necesita conexión ssh a todos los servers y permisos .
2- Se copia todo el directorio ansible-server para realizar las instalaciones y configuraciones de los playbooks

scp -r ansible-server -i /home/camila/.ssh/ec2_terraform ec2-user@ec2-34-239-104-156.compute-1.amazonaws.com:/home/ec2-user

Permisos:
chown -R ec2-user:ec2-user ansible-server
chmod 755 ansible-server

3- Con el script script_install_ansible.sh se instala la herramienta ansible y se copia la llave pública para la conectividad ssh al resto de los servidores.

Una vez instalado, se puede comenzar los playbooks de configuración.

Se usa un inventario dinámico debido a que la infraestructura se encuentra en AWS cloud y estas instancias al cambiarse y ejecutarse varía su dirección IP pública, debido a que la tiene autoasignada y no por Elastic IP

El servicio de Elastic IP no se usó debido a que presentan un costo adicional para la infraestructura en AWS.

Para el inventario dinámico se usó un plugin de aws que permite organizar los servers de acuerdo a sus tags de environment

Comando:
ansible-inventory -i inventory-dynamic/dev.aws_ec2.yml --graph

```
[ec2-user@ip-10-10-200-85 ansible-server]$ ansible-inventory -i inventory-dynamic/dev.aws_ec2.yml --graph
@all:
  |--@ungrouped:
  |--@aws_ec2:
  |  |--ec2-34-239-104-156.compute-1.amazonaws.com
  |  |--ec2-44-222-70-11.compute-1.amazonaws.com
  |  |--ec2-54-242-84-242.compute-1.amazonaws.com
  |  |--ip-10-10-1-79.ec2.internal
  |--@role_host:
  |  |--ec2-34-239-104-156.compute-1.amazonaws.com
  |--@role_frontend:
  |  |--ec2-44-222-70-11.compute-1.amazonaws.com
  |  |--ec2-54-242-84-242.compute-1.amazonaws.com
  |--@role_backend:
  |  |--ip-10-10-1-79.ec2.internal
[ec2-user@ip-10-10-200-85 ansible-server]$
```

Se crean roles para las instalaciones de diferentes paquetes en cada servidor que corresponda:
-nodejs
-mysql client
-jenkins server
-docker

Comando:
ansible-playbook -i inventory-dynamic/dev.aws_ec2.yml playbooks/frontend.yml

```
04-156.compute-1.amazonaws.com

TASK [nodejs : Install] ************************************************************************************
changed: [ec2-44-222-70-11.compute-1.amazonaws.com]
changed: [ec2-54-242-84-242.compute-1.amazonaws.com]

TASK [nodejs : Get npm version] ****************************************************************************
ok: [ec2-54-242-84-242.compute-1.amazonaws.com]
ok: [ec2-44-222-70-11.compute-1.amazonaws.com]

TASK [nodejs : Show npm version] ***************************************************************************
ok: [ec2-44-222-70-11.compute-1.amazonaws.com] => {
    "msg": "npm version: 10.8.2"
}
ok: [ec2-54-242-84-242.compute-1.amazonaws.com] => {
    "msg": "npm version: 10.8.2"
}

TASK [nodejs : install pm2] ********************************************************************************
changed: [ec2-44-222-70-11.compute-1.amazonaws.com]
changed: [ec2-54-242-84-242.compute-1.amazonaws.com]

PLAY RECAP *************************************************************************************************
ec2-44-222-70-11.compute-1.amazonaws.com : ok=5    changed=2    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0

ec2-54-242-84-242.compute-1.amazonaws.com : ok=5    changed=2    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
```

ansible-playbook -i inventory-dynamic/dev.aws_ec2.yml playbooks/backend.yml

ansible-playbook -i inventory-dynamic/dev.aws_ec2.yml playbooks/jenkinsserver.yml

```
camila@camila-IdeaPad-3-15ITL6:/media/camila/DATA/PROYECTO OMNICLOUD$ ssh -i /home/camila/.ssh/ec2_terraform ec2-user@ec2-34-239-1
04-156.compute-1.amazonaws.com
TASK [common : Actualizar servidor] ***********************************************************************************
ok: [localhost]

TASK [jenkins : Install Java version] *********************************************************************************
changed: [localhost]

TASK [jenkins : agregar repositorios de jenkins] **********************************************************************
changed: [localhost]

TASK [jenkins : Agregar jenkins llaves] *******************************************************************************
changed: [localhost]

TASK [jenkins : Install jenkins server] *******************************************************************************
changed: [localhost]

TASK [jenkins : Jenkins service] **************************************************************************************
changed: [localhost]

RUNNING HANDLER [jenkins : reload jenkins] ****************************************************************************
changed: [localhost]

PLAY RECAP ************************************************************************************************************
localhost                  : ok=7    changed=6    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0

[ec2-user@ip-10-10-200-85 ansible-server]$
```

Paso 3: Desplegar la app con jenkins server CI/CD

Con el servidor jenkins realizaremos un despliegue de la app en un entorno de testing por lo tanto podríamos hacerlo automático a los EC2, esto será Continuous Integration y Continuous Deployment

Instalación de Plugins necesarios instalados en el server Jenkins:

1- Github
2- Nodejs
3- Docker pipeline
4- SSH Agent

Guardar credenciales en Jenkins

Ve a: Manage Jenkins → Credentials

Agrega:

- Tipo: **Username with password**
- ID: rds-db-credentials

- Username: (tu user RDS)

- Password: (tu password RDS)

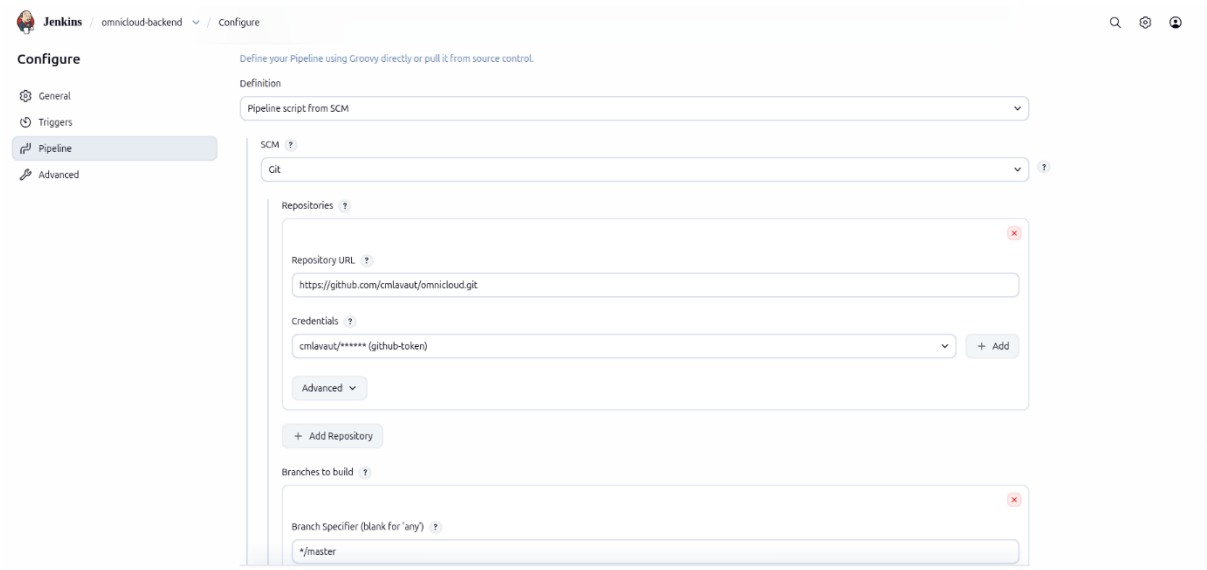
De esta manera agregar credenciales de:

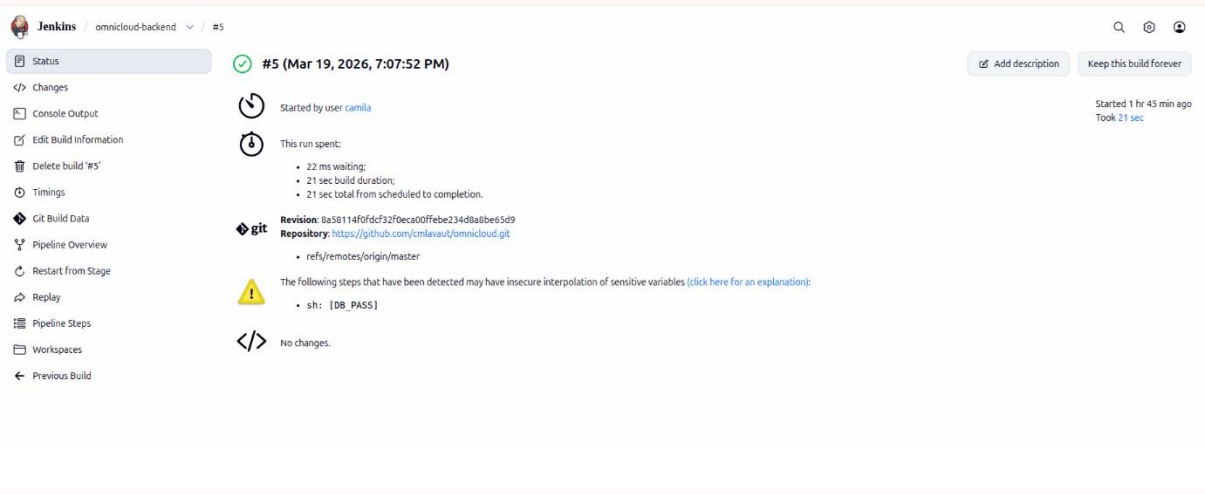
Amazon RDS	ID: rds-db-credentials
Github Token	ID: github
EC2 credentials	ID: ec2
DockerHub credentials	ID: dockerhub-credentials

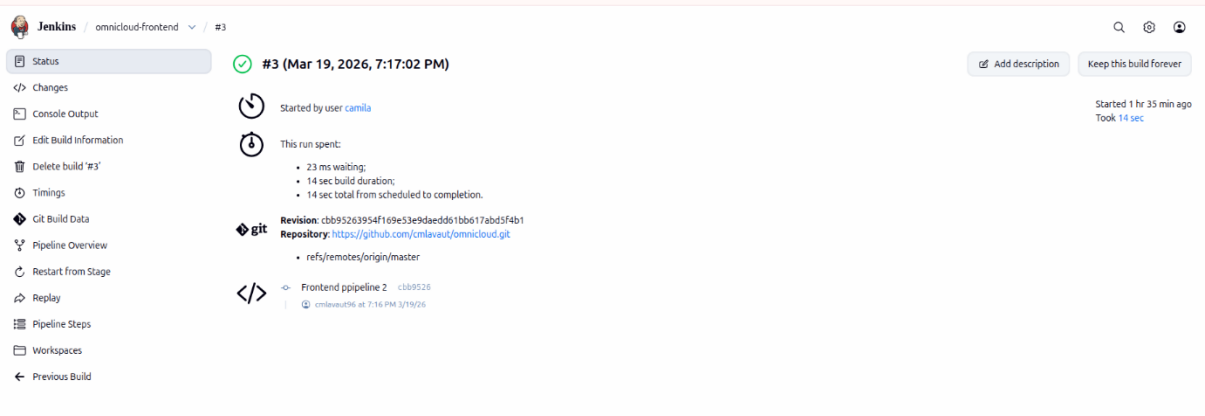
Antes de crear los pipelines nos conectamos desde el hostjump a la instancia RDS y creamos las tablas de la base de datos movie_db para la aplicación.

```
MySQL [movie_db]> show tables;
+--------------------+
| Tables_in_movie_db |
+--------------------+
| movies             |
| publications       |
| reviewers          |
+--------------------+
3 rows in set (0.002 sec)

MySQL [movie_db]>
```

Posteriormente se crear los jobs de jenkins basados en Jenkinsfile declarativos correspondientes al frontend y backend, para el despliegue de la app con contenedores dockers.

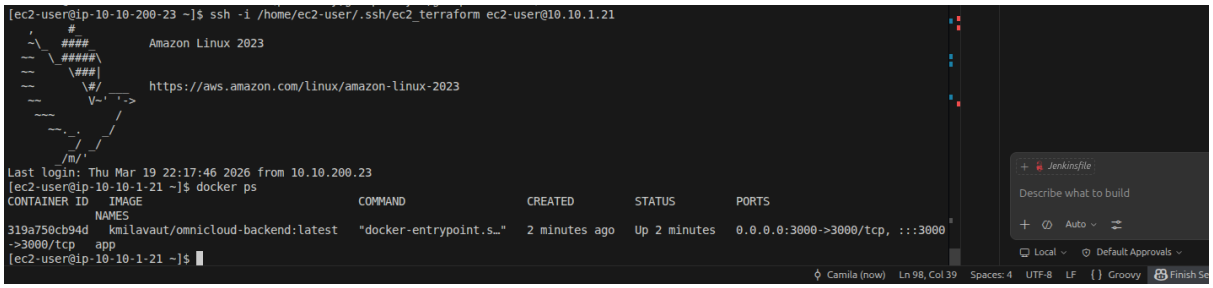






En los pipelines se encuentran los stages de build, test, push en dockerhub, deploy en EC2 e insertar los datos en la base de datos. (Código insertado)

Se adjunta captura de los contenedores de la app funcionando en cada una de las EC2 de frontend y backend y de la app a través del DNS del Load Balancer.



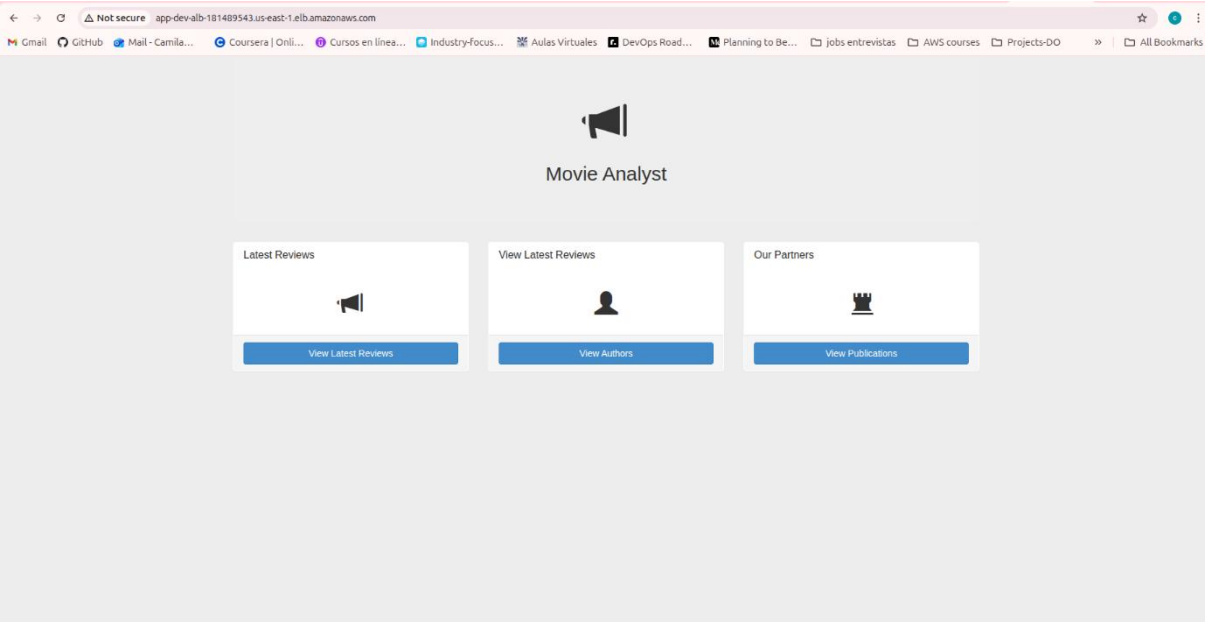
```
camila@camila-IdeaPad-3-15ITL6:/media/camila/DATA/PROYECTO OMNICLOUD$ ssh -i /home/camila/.ssh/ec2_terraform.pub ec2-user@ec2-44-2
07-0-246.compute-1.amazonaws.com

Database changed
MySQL [movie_db]> show tables;
+--------------------+
| Tables_in_movie_db |
+--------------------+
| movies             |
| publications       |
| reviewers          |
+--------------------+
3 rows in set (0.002 sec)

MySQL [movie_db]> SELECT * FROM publications;
+----+----------------------------+----------------------+
| id | name                       | avatar               |
+----+----------------------------+----------------------+
|  1 | The Daily Reviewer         | glyphicon-eye-open   |
|  2 | International Movie Critic | glyphicon-fire       |
|  3 | MoviesNow                  | glyphicon-time       |
|  4 | MyNextReview               | glyphicon-record     |
|  5 | Movies n' Games            | glyphicon-heart-empty |
|  6 | TheOne                     | glyphicon-globe      |
|  7 | ComicBookHero.com          | glyphicon-flash      |
+----+----------------------------+----------------------+
7 rows in set (0.001 sec)

MySQL [movie_db]> SELECT * FROM movies;
+----+----------------------------+--------------+-------+----------------+----------------------------+
| id | title                      | release_year | score | reviewer       | publication                |
+----+----------------------------+--------------+-------+----------------+----------------------------+
|  1 | Suicide Squad              | 2016         |     8 | Robert Smith   | The Daily Reviewer         |
|  2 | Batman vs. Superman        | 2016         |     6 | Chris Harris   | International Movie Critic |
|  3 | Captain America: Civil War | 2016         |     9 | Janet Garcia   | MoviesNow                  |
|  4 | Deadpool                   | 2016         |     9 | Andrew West    | MyNextReview               |
|  5 | Avengers: Age of Ultron    | 2015         |     7 | Mindy Lee      | Movies n' Games            |
|  6 | Ant-Man                    | 2015         |     8 | Martin Thomas  | TheOne                     |
|  7 | Guardians of the Galaxy    | 2014         |    10 | Anthony Miller | ComicBookHero.com          |
|  8 | Doctor Strange             | 2016         |     7 | Anthony Miller | ComicBookHero.com          |
|  9 | Superman: Homecoming       | 2017         |    10 | Chris Harris   | International Movie Critic |
| 10 | Wonder Woman               | 2017         |     8 | Martin Thomas  | TheOne                     |
+----+----------------------------+--------------+-------+----------------+----------------------------+
10 rows in set (0.001 sec)

MySQL [movie_db]>
```

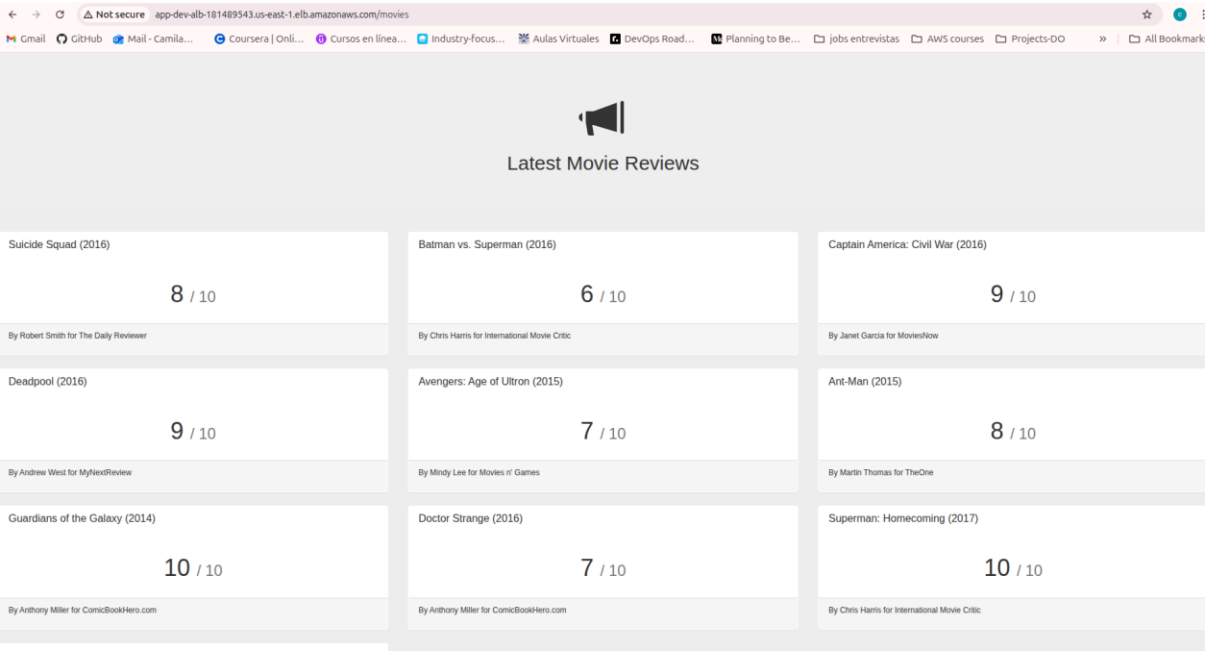
bash
ssh

Build with Agent
AI responses may be inaccurate
Generate Agent Instructions to onboard AI onto your codebase.

Jenkinsfile
Describe what to build
Auto
Local
Default Approvals

App conexión mediante el Application Load Balancer ejecutado en la infraestructura





Casos Prácticos

Responder brevemente cómo actuaría ante los siguientes escenarios:

1- Falla de un despliegue en producción

Primeramente, se analiza los logs del backend, contenedores y base de datos:
docker logs namecontenedor
docker exec -it nombrecontenedor –bash y cd /var/log
revisar metricas de (CPU, memoria, errores HTTP) para identificar la causa:
htop para procesos de CPU, free -m
pm2 logs para los logs de la app nodejs o tail -f logs/app.log
journalctl -u <service> para logs del sistema

Adicionalmente se realiza un rollback a la última versión estable usando el pipeline (CI/CD) y se peuden revisar las metricas de la herramienta de monitoreo en tiempo real como AWSCloudwatch o Prometheus.

Otra opción de chequeo es verificar variables de entorno, conexiones (ej. MySQL/RDS), puertos y dependencias.

netstat -tulnp | grep 3000 para ports en caso de mi app

2- Falta de espacio en disco en un servidor

Primeramente, se necesita identificar qué está consumiendo espacio:
df -h

Ubicar archivos grandes o crecimiento anormal (logs en lo general)
du -ah / | sort -rh | head -20

Teniendo esta información se pueden limpiar logs antiguos, eliminar archivos temporales o rotar logs (logrotate)

Si se trabaja con Docker se pueden eliminar imágenes/volúmenes no usados
docker system prune -a

En caso de que el comportamiento sea recurrente, ampliar disco (EBS) o rediseñar almacenamiento

3- Expiración de certificados TLS

De primera acción se verifica la fecha de expiración

Si se usa Lets Encrypt renovar el certificado con: certbot renew o regenerar desde el proveedor.

Reload del servicio: systemctl reload nginx o systemctl reload apache2

4- Validación de respaldos

Ejecutar backups de forma periódica y realizar sus validaciones:
Base de datos: mysqldump -u applicationuser -p movie_db > backup.sql
Automatizarlos con CI/CD pipelines en cronjobs

En AWS (muy recomendado)

- Si usas RDS, AWS se encarga del backup
- AWS Backup: Centraliza backups de múltiples servicios
- S3: Guardar backups como archivos (backup.sql)

Luego de los backups realizar validación de integridad (checksums)
md5sum backup.sql